

Multi-Tier Computing (19Z011)

Comprehensive Exam Preparation Notes

PSG College of Technology · CSE · 2024–25

Synthesised from Team presentations: EISS (Team 8), Client Components (Team 2), E-Commerce Architecture (Team – E-Commerce), Sales Automation (Team 10), and Team 11

How to use these notes for your exam: Every question that asks you to “*design a multi-tier architecture for [scenario]*” follows the same pattern. Apply the three-tier template: **(1) Presentation Tier** — what the user sees and does; **(2) Application Tier** — business logic, rules, workflows, APIs; **(3) Data Tier** — what gets stored, how, and how it is retrieved. For each tier, always address: *components, key functions, technologies, and how it connects to the adjacent tier.*

Contents

1	Use Case 1 — Enterprise Information Sharing System (EISS)	2
1.0.1	Security in EISS	3
1.0.2	Performance Considerations for EISS	3
1.0.3	Governance in EISS	4
2	Use Case 2 — Client Components in a Front-End / Food Delivery App	4
2.0.1	Client Components (Presentation Tier Concepts)	4
2.0.2	Key Features of Modern Front-End Tools	4
2.0.3	Food Delivery App – Applied Three-Tier Design	5
2.0.4	Case Study: Conflicts and Solutions (Important for Exam)	5
3	Use Case 3 — 3-Tier Architecture in E-Commerce Systems	5
3.0.1	Case Study A: Flipkart	7
3.0.2	Case Study B: Zomato	7
4	Quick Revision Summary	8

Use Case 1 — Enterprise Information Sharing System (EISS)

(Source: Team 8 – Pratish Mithra J, Sandeep K, Rohith Prakash)

Scenario: A platform enabling employees, managers, departments, and external stakeholders to create, exchange, search, update, and govern information across an organisation. Supports document management, knowledge sharing, workflow approvals, reporting, partner collaboration, case tracking, and operational decision-making.

EISS – Presentation Tier

Interfaces: Web Portal (Browser UI), Mobile App (iOS/Android), Admin Console (Policy & Control), Partner/API Client (External Access).

Key Functions:

- **UI Rendering** – Displays enterprise data via dashboards, forms, and reports in a user-friendly format.
- **User Input Handling** – Captures queries, file uploads, and form submissions; forwards them to the Application Tier.
- **Client-Side Validation** – Basic checks (required fields, format) before sending data to reduce backend load.
- **API Communication** – Communicates with backend services via REST/GraphQL APIs.
- **Role-Based Views** – Ensures users see only information relevant to their role (personalisation).

EISS – Application Tier

Components: API Gateway (Auth, routing, rate limits), Business Services (Workflow and rules), Collaboration Services (Sharing, search, alerts), Integration Services (ERP, CRM, email, SSO).

Key Functions:

- **Business Logic Processing** – Implements organisational rules: approval workflows, access policies, data-sharing protocols.
- **API Management** – Exposes APIs; API gateway manages routing, authentication, and request handling.
- **Authentication & Authorisation** – Identity verification and RBAC ensure only authorised users access specific data.
- **Workflow Management** – Document approvals, notifications, and task assignments.
- **Integration** – Connects to ERP, CRM, HR systems, email and notification services.
- **Data Processing & Transformation** – Raw data from Data Tier is processed, filtered, and formatted before reaching the Presentation Tier.

EISS – Data Tier

Components: Operational DB (Users, records, metadata), Document Store (Files and content), Search Index (Fast retrieval), Audit and Backup (Logs, retention).

Key Functions:

- **Data Storage** – Stores user records, documents, transaction logs, and metadata (structured and unstructured).
- **Data Retrieval** – Efficiently retrieves data based on queries from the Application Tier.
- **Data Integrity & Consistency** – Constraints, transactions, and normalisation ensure accuracy.
- **Backup & Recovery** – Regular backups prevent data loss and ensure business continuity.
- **Logging & Auditing** – All activities recorded for security, compliance, and monitoring.
- **Search & Indexing** – Search engines and indexing enable fast retrieval of large volumes of data.

Security in EISS

Authentication:

- Username/password
- Multi-Factor Authentication (MFA)
- Single Sign-On (SSO)

Data Encryption:

- In transit: HTTPS/TLS
- At rest: Database encryption

Authorisation:

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)

API Security:

- API Gateway enforces authentication
- Rate limiting prevents abuse
- Token-based access (JWT, OAuth)

Auditing & Logging: Tracks user activity, ensures compliance, helps detect anomalies.

Performance Considerations for EISS

- **Scalability** – Horizontal scaling (adding servers); Vertical scaling (upgrading resources).
- **Load Balancing** – Distributes incoming traffic across multiple servers to avoid overload.
- **Caching** – Application-level caching; CDN caching for static content.
- **Database Optimisation** – Indexing, query optimisation, partitioning.
- **Asynchronous Processing** – Background jobs; Message queues (Kafka, RabbitMQ).

Governance in EISS

- **Data Governance** – Data ownership; data classification (sensitive, confidential); data lifecycle management.
- **Access Policies** – Who can access what; role-based restrictions.
- **Compliance Standards** – GDPR, HIPAA, and industry regulations.
- **Audit & Accountability** – Track all system activities; maintain logs for audits.

Use Case 2 — Client Components in a Front-End / Food Delivery App

(Source: Team 2 – G Elakkiya, B Snesha, S Tharigalakshmi, R Varshini, V Dharshini)

Scenario: A startup builds an online food delivery web application (similar to Swiggy/Zomato). Users can search restaurants, view menus, add items to cart, place orders, and track delivery status.

Client Components (Presentation Tier Concepts)

What are Client Components?

Parts of an application that run on the user's device and directly interact with the user. In multi-tier computing, client components form the **Presentation Layer**.

Characteristics:

- **Platform Independent** – runs on different devices and browsers.
- **Responsive** – adapts to various screen sizes.
- **Interactive** – provides real-time feedback.
- **Lightweight** – focuses mainly on presentation logic.

Handles: User interactions (clicks, inputs, navigation); displaying content from the server; basic processing before sending data to the backend.

Key Features of Modern Front-End Tools

- **Component-Based Architecture** – Reusable components; faster development, easier maintenance (React, Angular, Vue).
- **Virtual DOM** – Updates only changed parts of the webpage; improves speed.
- **Responsive Design Support** – Interfaces work across desktops, tablets, and mobiles.
- **State Management** – Manages application data efficiently across components (Redux).
- **Package Management** – npm (most widely used), Yarn (faster, reliable).
- **Build Tools & Bundlers** – Webpack (bundles files), Vite (faster dev), Parcel (auto-optimises).
- **Version Control** – Git & GitHub for tracking changes and collaboration.
- **Dev & Debugging Tools** – VS Code, Chrome DevTools.

- **API Integration** – Enables communication with backend services for dynamic data.
- **Cross-Browser Compatibility** – Consistent behaviour across all browsers.

Food Delivery App – Applied Three-Tier Design

Presentation Tier Components (Food Delivery App)

- Search Component – helps users find restaurants.
- Restaurant List Component – shows available restaurants.
- Menu Component – displays food items.
- Cart Component – stores selected items.
- Order Tracking Component – shows delivery status.

All components update the page without full page refresh (Single Page App behaviour).

Case Study: Conflicts and Solutions (Important for Exam)

Conflict 1: Lack of Proper State Management

Symptom: Cart did not update immediately when items were added.

Cause: Each client component managed its own data separately.

Impact: Users thought cart was broken; items added multiple times; poor UX.

Solution: Centralised state management using **Redux**. Cart updates instantly; all components share the same data; no page refresh needed.

Conflict 2: Slow Interface Performance

Symptom: Web page loaded all restaurants, menus, and images at once.

Cause: No performance optimisation in front-end tools.

Impact: Slow loading pages, high bounce rate, poor mobile performance.

Solution: Used Webpack with **code splitting**, **lazy loading**, and **file compression**. Result: faster load times, reduced file size, improved mobile performance.

Conflict 3: Difficulty Managing Large UI Code

Symptom: Front-end code became large and unmaintainable as features grew.

Cause: Developers wrote the UI as one large script instead of modular components.

Impact: Hard to debug, difficult to reuse code, slow development.

Solution: Reorganised using **component-based architecture** with React. Separate reusable components: Navbar, Restaurant Card, Menu, Cart.

Use Case 3 — 3-Tier Architecture in E-Commerce Systems

(Source: Team – E-Commerce – Dwarkesh, Kapil Arif, Sudharsan, Vasanthkumaar, Thakshin Kumar T, Akash S)

Scenario: A scalable, maintainable, and secure online retail platform (e.g., a system like Flipkart or Zomato).

E-Commerce – Presentation Tier

The only layer customers directly interact with – via web browsers or mobile apps.

Functions:

- **Product Discovery** – Search bars, filters, product listings, sorting & category navigation.
- **Shopping Cart & Checkout** – Cart management, address forms, order summary, payment UI.
- **Account Management** – Login/signup, profile pages, order history, wishlist.
- **Multi-Platform Reach** – Responsive web design + native iOS/Android apps.

Why it matters: Independent scalability during flash sales; a 1-second delay in page load reduces conversions by 7%; UI redesigns require zero changes to business logic; no direct DB access eliminates injection attacks.

E-Commerce – Application Tier

The intelligence of the system. Sits between UI and database; processes every user action with business rules.

Functions:

- **Order Processing** – Receives order requests, validates, and coordinates fulfillment steps.
- **Payment Validation** – Verifies payment details, interfaces with payment gateways, handles fraud checks.
- **Inventory Management** – Checks stock levels in real time, reserves items, triggers replenishment.
- **Pricing & Promotions** – Applies discount codes, dynamic pricing rules, taxes, and shipping calculations.
- **Centralised Business Rules** – Discount logic, tax policies, and shipping rules in one place; updates affect all channels.
- **Concurrency Handling** – Thousands of simultaneous transactions via load balancing, queuing, request routing; prevents inventory race conditions.
- **Security Firewall** – UI can NEVER query the DB directly; all access is mediated, logged, and controlled here.
- **Integration Hub** – Connects to payment gateways (Stripe, PayPal, Razorpay), shipping APIs (FedEx, UPS), and analytics.

E-Commerce – Data Tier

The permanent memory of the system. Only responds to queries from the Application Tier; never exposed to the outside world.

Data Stored:

- **Product & Inventory DB** – SKUs, descriptions, pricing, stock levels; updated in real time.

- **Customer & Account Data** – Profiles, addresses, credentials (hashed), purchase history.
- **Order & Transaction Records** – Every order, status, payment confirmation, fulfilment history, refunds.
- **Session & Cache Layer** – Redis/Memcached for high-speed access to frequently queried data.

Technology Stack (Exam-ready):

DB Type	Technology	Used For
Relational (SQL)	MySQL / PostgreSQL	Orders, customers, payments – ACID-compliant transactional data
NoSQL	MongoDB / DynamoDB	Product catalogs, user sessions, flexible/evolving schemas
Cache	Redis / Memcached	Lightning-fast reads for popular products, sessions, search results
Search Engine	Elasticsearch	Full-text search, faceted filtering, auto-complete

Case Study A: Flipkart

Flipkart – Technology Stack by Tier

Presentation: React Native (Android/iOS), React.js (Web), Akamai CDN, Flipkart Lite PWA (for low-bandwidth users).

Application: Java Microservices, Apache Kafka, Kubernetes, Razorpay/PhonePe/UPI APIs. 100+ microservices handle search ranking, recommendations, fraud detection, and seller notifications independently.

Data: MySQL (transactional ACID), Apache HBase, Elasticsearch (15 crore+ listings), Redis (Deals of the Day cache), Apache Cassandra (event logs, clickstream, seller analytics).

Challenge → Solution Mapping:

- **10 crore visitors in 1 hour (Big Billion Days)** → Only Application Tier (Kubernetes pods) auto-scales; DB and storefront unchanged.
- **15 crore+ searchable product listings** → Elasticsearch indexes all; returns ranked results in milliseconds without touching main DB.
- **Multiple UIs (App, Web, Flipkart Lite)** → Four separate Presentation Tiers all connect to the same Application and Data Tiers via API.
- **UPI payment must never charge twice** → Application Tier checks DB for transaction status before retrying; MySQL ACID compliance guarantees single deduction.

Case Study B: Zomato

Zomato – Technology Stack by Tier

Presentation: React Native (Android/iOS), React.js (Web), Cloudflare CDN. Storefront API enables web version and partner integrations.

Application: Ruby on Rails, Go Microservices, Kafka, Kubernetes, Razorpay/PayTM APIs. Core logic: order validation, delivery partner assignment, ETA calculation, surge pricing, fraud checks.

Data: MySQL (ACID orders/payments), Redis (Top Restaurants Near You cache), Elasticsearch (typo-tolerant search), Cassandra (event logs), Google Cloud Storage (photos/menus).

Challenge → Solution Mapping:

- **10x orders in 10 minutes (New Year's)** → Only Application Tier (Kubernetes) auto-scales; zero user-facing slowdown.
- **5 lakh restaurants with different menus daily** → NoSQL stores flexible data; Elasticsearch makes it searchable with typo-tolerance and cuisine filters.
- **Android, iOS, Website – all different UIs** → Three separate Presentation Tiers connect to the same backend via API.
- **UPI payment must never double-debit** → Application Tier checks DB before retry; MySQL ACID guarantees single deduction.

Quick Revision Summary

	Presentation Tier	Application Tier	Data Tier
Role	User Interface	Business Logic	Storage
Who uses it	End users, admins, partners	No one directly	No one directly
Key tech	React, Angular, CDN	REST APIs, Kafka, Kubernetes	MySQL, Redis, Elasticsearch
Scales when	High UI traffic	High transaction volume	High read/write volume
Security	Input validation, HTTPS	Auth, RBAC, API gateway	Encryption, access control
EISS example	Web portal, Admin console	Workflow engine, API Gateway	Operational DB, Document store
E-Commerce example	Product pages, Cart UI	Order processor, Payment validator	Product DB, Order DB, Cache
Food Delivery example	Restaurant search, Tracking map	Order routing, ETA calculation	Menu store, Order records

Final Exam Tip: For any scenario, the answer structure is always:

Introduction (why 3-tier for this scenario) → **Presentation Tier** (UI components + technologies) → **Application Tier** (business rules + APIs + integrations) → **Data Tier** (storage types + technologies) → **Request Flow** (numbered steps) → **Security** → **Performance** → **Advantages**.